

What Predicts Software Developers' Productivity?

Nikhil Anand

February 18, 2026

Organizations have a variety of options to help their software developers become their most productive selves, from modifying office layouts, to investing in better tools, to cleaning up the source code. But which options will have the biggest impact? Drawing from the literature in software engineering and industrial/organizational psychology to identify factors that correlate with productivity, this paper designed a survey that asked 622 developers across 3 companies about these productivity factors and about self-rated productivity.

1 Motivation

Improving productivity of software developers is important. By definition developers who have completed their tasks can spend their freed-up time on other useful tasks, such as implementing new features or on new verification and validation activities. Organizations such as ours demand empirical guidance on which factors to try to manipulate in order to best improve productivity. For example, should an individual developer (1a) spend time seeking out the best tools and practices, or (1b) shut down email notifications during the day? Should a manager (2a) invest in refactoring to reduce code complexity, or (2b) give developers more autonomy over their work? Should executives (3a) invest in better software development tools, or (3b) should they invest in less distracting office space? In an ideal world, we would invest in a variety of productivity-improving factors, but time and money are of productivity-improving factors, but time and money are limited, so we must make selective investments.

The paper focuses on answering three questions:

1. What factors most strongly predict developers' self-rated productivity
2. How do these factors differ across companies?
3. What predicts developer self-rated productivity, in particular, compared to other knowledge workers?

2 Methodology

The authors designed a large-scale survey study to investigate what factors predict software developers' self-rated productivity. Their methodological approach was primarily **quantitative and correlational**, relying on regression analysis across multiple companies. The central goal was breadth: instead of deeply examining one or two factors, they aimed to evaluate a wide spectrum of potential productivity predictors drawn from both software engineering and industrial/organizational psychology literature.

Measuring Productivity. But How? A foundational methodological decision was how to measure productivity. Rather than using objective metrics such as lines of code or number of commits alone, the authors chose a subjective measure: *self-rated productivity*¹. Participants were asked to rate their agreement with the statement, "*I regularly reach a high level of productivity.*" The researchers justified this choice by arguing that objective measures can be gamed, incomplete, or misleading (for example, a

¹Productivity here does not mean *how productive one is?* but rather *how productive they are consistently?*

small but critical bug fix might reflect high productivity despite minimal code written). Self-ratings, although imperfect, allow respondents to integrate multiple dimensions of their work experience, including output quality, efficiency, and workflow effectiveness. To validate this decision, they conducted a contextual analysis correlating self-rated productivity with objective measures (lines changed and changelists merged) within Google. These objective measures showed statistically significant but very small explanatory power ($R^2 < 3\%$)², reinforcing that subjective productivity captures broader aspects than raw output metrics.

What are the factors that decide productivity? The next major methodological component involved identifying candidate productivity factors. The authors began with 127 potential factors derived from four primary literature sources: prior structured reviews of knowledge worker productivity, validated survey instruments from organizational psychology (such as the Work Design Questionnaire), a systematic review of software engineering productivity factors, and prior empirical work on developers' perceived productivity. They also added three additional factors of particular organizational interest. To reduce survey fatigue and ensure practicality, they applied three filtering criteria: eliminating duplicates, condensing similar constructs, and favoring factors with practical utility. This iterative, collaborative refinement process ultimately reduced the pool to 48 survey items. Each factor was presented as a Likert-scale agreement statement (Strongly disagree to Strongly agree).

2.1 The Experiment

The study was conducted across three companies: Google, ABB, and National Instruments. This multi-company design strengthened external validity by testing whether relationships generalized across different organizational contexts (e.g., open vs. mixed office layouts, monolithic vs. distributed repositories, centralized vs. flexible tooling). Each company administered the same core survey, though small contextual adjustments were made when necessary. In total, the final valid sample included 407 Google developers, 137 ABB developers, and 78 National Instruments developers.

To address their third research question, *how developers differ from other knowledge workers*, the authors also deployed a modified version of the survey to Google analysts. Software-specific questions were removed or adapted. This comparison group allowed the researchers to determine whether certain productivity predictors were uniquely strong for developers or more broadly applicable to knowledge workers.

The survey also collected demographic variables — gender, tenure, and seniority — which were treated as control variables in the regression models. These were included to reduce confounding effects. For example, earlier analysis showed that more senior developers tended to rate themselves slightly less productive, making seniority an important covariate. Missing demographic data were imputed conservatively (e.g., using mean or most common values), since demographics were not primary variables of interest but controls.

To ensure data quality, the researchers included an attention check item instructing respondents to select a specific answer. Surveys failing this check were discarded. This step removed 6–22% of responses depending on the company, increasing confidence that retained responses reflected careful consideration.

The core analytical strategy involved running separate multiple linear regression models for each factor at each company. In each model, self-rated productivity served as the dependent variable, and one productivity factor served as the independent variable, while demographic variables were included as controls. Importantly, the authors ran 48 separate regressions per company rather than building a single multivariate model including all factors simultaneously. This allowed them to isolate the strength of association for each factor individually while preserving company data privacy. Because running many statistical tests increases the risk of false positives, they corrected p-values using the Benjamini–Hochberg method to control the false discovery rate.

In interpreting results, the authors emphasized effect size (regression coefficient magnitude) more than statistical significance alone. They noted that statistical significance is sensitive to sample size (Google's

²In regression analysis, R^2 (R-squared) tells us how much of the variation in the outcome variable can be explained by the predictor(s). It ranges from 0 to 1

larger sample produced more significant results), whereas effect size better captures practical impact. They also analyzed variance across companies to assess generalizability and compared coefficient differences between developers and analysts to detect role-specific predictors.

2.2 Causality Limitations

The authors are very clear that their methodology identifies correlations, not causal relationships. That distinction is crucial. The survey measures how strongly each factor (like job enthusiasm or feedback quality) is statistically associated with self-rated productivity. But association does not automatically mean that the factor causes productivity changes.

1. Correlation Does Not Establish Direction The biggest limitation is directionality. For example, the strongest predictor in the study is job enthusiasm. The data show that higher enthusiasm is associated with higher self-rated productivity. But which direction is the arrow?

- Does enthusiasm cause productivity? $\text{enthusiasm} \rightarrow \text{productivity}$
- Or does being productive make you feel enthusiastic? $\text{productivity} \rightarrow \text{enthusiasm}$
- Or is there a bidirectional relationship, where enthusiasm and productivity reinforce each other? $\text{enthusiasm} \leftrightarrow \text{productivity}$

The survey design cannot distinguish between these possibilities because all variables were measured at the same time (cross-sectional design). There is no time-based sequencing that would allow us to infer “A happened before B.”

2. Potential Third Variables (Confounding Factors) Another causality threat is the presence of unmeasured third variables. A third factor might influence both the predictor and productivity simultaneously. For example

- Strong leadership culture could increase both useful feedback and productivity.
- Organizational stability might increase both enthusiasm and performance.
- Personal traits (e.g., conscientiousness) might make someone both more productive and more likely to report positive perceptions.

If such confounders are not measured or controlled for, the regression coefficient may reflect indirect relationships rather than direct causal effects. The authors controlled for demographics (gender, tenure, seniority), but many psychological or organizational traits were not controlled.

3. Causality Evidence Depends on Prior Literature The authors acknowledge that their confidence in causality partly depends on prior research.

- There is strong meta-analytic evidence from organizational psychology that feedback improves performance.
- There is experimental evidence in other contexts that autonomy increases motivation and output.

So for some factors, there is theoretical and empirical support for causal direction. However, not all 48 factors have equally strong causal backing. The paper does not conduct a systematic meta-analysis to establish causal strength for each factor. Therefore, the strength of causal inference varies across factors.

3 Key Findings

RQ 1: What factors most strongly predict developers’ self-rated productivity? The central finding of the study is that the strongest predictors of self-rated productivity were non-technical, human-centered factors, not technical or platform-related ones. After running regression models across three companies, the authors identified three factors that were statistically significant predictors across all

organizations - (1) Job enthusiasm, (2) Peer support for new ideas, (3) Receiving useful feedback about job performance.

Among these, job enthusiasm had the strongest overall effect size. This means that developers who reported being more enthusiastic about their jobs were substantially more likely to rate themselves as highly productive.

A particularly striking result is that the top-ranked predictors were overwhelmingly social and psychological, rather than technical. Factors such as architecture complexity, processing power requirements, or platform volatility — many of which are emphasized in traditional productivity models like COCOMO II³ — ranked much lower in predictive strength.

This directly answers RQ1 by showing that individual-level productivity is more strongly associated with - motivation and emotional engagement, team climate and support and feedback and recognition mechanisms rather than purely technical characteristics of the software or development environment.

RQ 2: How do these productivity factors differ across companies? The study compared results across Google, ABB, and National Instruments to examine consistency and variation. Some factors showed remarkable stability across organizations, particularly, peer support for new ideas, useful feedback and the use of remote work for concentration. These factors had low variance across companies, suggesting they may generalize well beyond the specific organizations studied.

However, some factors varied substantially across companies, including, the use of best tools and practices, code reuse and the accuracy of incoming information. For example, the importance of using the best tools was much stronger at Google than at National Instruments. The authors suggest organizational structure may explain this — such as Google’s large monolithic codebase making tool effectiveness more critical.

Thus, RQ2 is answered by showing that

- Human and social factors tend to generalize across organizations
- Technical and tooling-related factors are more context-dependent

This highlights that productivity drivers are not universally identical; organizational environment matters.

RQ 3: What predicts developer productivity in particular, compared to other knowledge workers? To address this, the authors compared Google developers with Google analysts (a non-developer knowledge worker group). They found both similarities and differences.

Similarities The authors found that job enthusiasm predicted productivity strongly for both groups. Some social and autonomy-related factors were important for both developers and analysts.

Differences Developers’ productivity was more strongly related to task variety and their ability to work effectively away from their desks. In contrast, analysts’ productivity was more strongly associated with positive feelings toward teammates and their time management autonomy.

The authors interpret this as evidence that:

- Developers may be especially sensitive to interruption and context switching.
- The ability to work away from distractions may be particularly important for programming work.
- Task variety may matter more for developers due to shared toolsets and reduced switching costs.

Thus, RQ3 is answered by showing that while developers share many productivity drivers with other knowledge workers, certain factors — especially around focus and task structure — are uniquely important in software development.

³COCOMO II model, Constructive Cost Model II, is a widely used, updated, and flexible algorithmic model designed to estimate software project cost, effort, and schedule - <https://softwarecost.org/tools/COCOMO/>

4 Implications

1. Productivity investments should prioritize human factors The most immediate implication is that organizations may be over-investing in technical optimizations while under-investing in social and psychological factors. Since the strongest predictors of productivity were job enthusiasm, peer support for new ideas and receiving useful feedback, companies aiming to improve developer productivity should prioritize (1) increasing engagement and motivation, (2) creating psychologically safe environments, and (3) improving feedback systems

This suggests that productivity initiatives might be more effective if they focus on culture and management practices rather than solely on tooling or architecture improvements.

2. Developer productivity is not just about tools Although tools and practices matter, they were not consistently the strongest predictors across companies. In some organizations (like Google), tooling had stronger relationships; in others, it did not. Tooling investments may yield diminishing returns if cultural factors are weak. Organizations should evaluate whether morale, autonomy, or feedback systems need attention before investing heavily in technical infrastructure.

In short, improving productivity is not just a tooling problem — it's a leadership and organizational design problem.

3. Feedback systems are strategically important Useful feedback about job performance was one of the few factors consistently significant across all companies. Companies should design structured, constructive feedback processes and ensure feedback is timely and behavior-focused, while also avoiding vague or purely evaluative performance reviews. Feedback is not just an HR mechanism — it is a productivity driver.

4. Software engineering research may be overly technical The finding that non-technical factors dominate productivity prediction suggests that software engineering research may need to shift focus. Traditionally, productivity studies emphasize - (1) code complexity, (2) architecture, and (3) platform volatility.

But this study suggests that organizational psychology and human factors may yield greater practical impact and thus, cross-disciplinary research between SE and I/O psychology is needed.

5. Rethinking traditional productivity models Many traditional models (like COCOMO II) emphasize project-level technical factors. This study suggests that those models may not translate well to individual-level productivity. Individual productivity is more socially and psychologically influenced than those models account for. Future productivity models may need to integrate human and cultural dimensions more explicitly.